

ELEC 328 Project Report

[Assn 1 – Procedural Mesh Interactions]

[Jonathan Hinojosa] – [S01460164] – [jh218]

Honor Statement: On my honor, I have neither given nor received any unauthorized aid on this project.

-Jonathan Hinojosa

1. Concept



The objective of this assignment was to create a scene that contained objects and scripts created by the student to demonstrate their knowledge of vertex-level deformations. In my project, (blandly) titled ProceduralMeshInteractions-ELEC328, I created the scene titled PMI-Jhino, which contains 4 objects of note for this assignment: 2 spheres, 1 flattened sphere, and a dog. The idea was to create a basic scene where a dog was chasing a ball it would eventually catch and an event would occur. I created 3 vertex-level deformation scripts (2 continuous, 1 conditional), 1 rotation script, and 1 collision detection script to demonstrate my understanding of vertex deformations.

2. Technical overview

CosGrow.cs and SinGrow.cs:

For part of these two scripts relevant to the project, I created a system which causes the Dog and ColliderBall objects to both grow with respect to time. The way I achieved this was by creating a new modifiedVertices Vector3 object, implementing a scaleFactor float using a $\text{Mathf.Sin()}/\text{Mathf.Cos()}$ function, using that scaleFactor to create a Vector3 axisScale object, and iterating over the base vertices to multiply them by the axisScale. This ultimately, vertex-by-vertex, changed the scale of the Dog and ColliderBall continuously.

TimeRipple.cs:

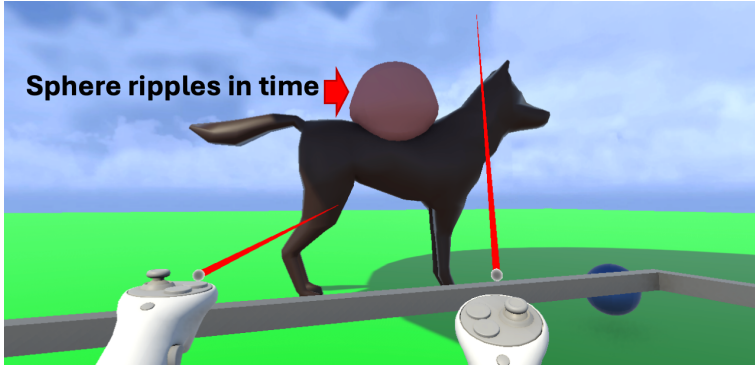
For the part of this script relevant to the project, I created a script to continuously push vertices outward to create a continuous ripple effect. This was achieved by first defining a few input values: amplitude to increase the height of the ripples, frequency to increase the density of the ripples, speed to increase the speed at which the ripples moved over the surface, and localEpicenter to determine where the ripple originates from on the object. Using the localEpicenter, a for loop determines the vertices closest to the input localEpicenter, creates a wave float object which uses the Mathf.Sin function, the distance from the epicenter, and time, and adds the wave float to each vertex, causing each vertex to increase relative to their position from the epicenter continuously. This effect is used on the Sphere object to create a continuous ripple effect.

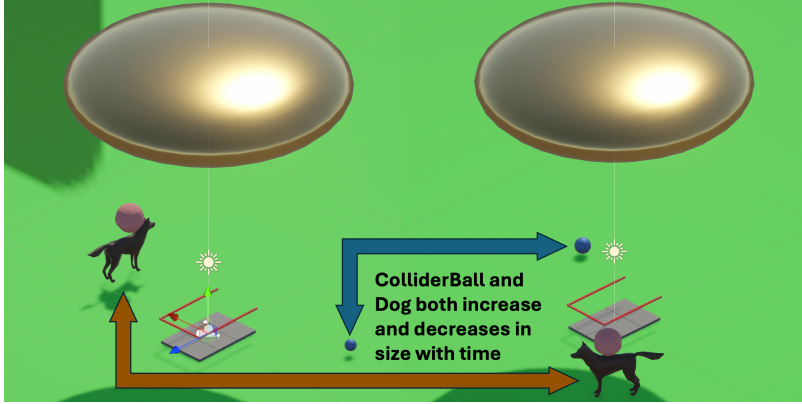
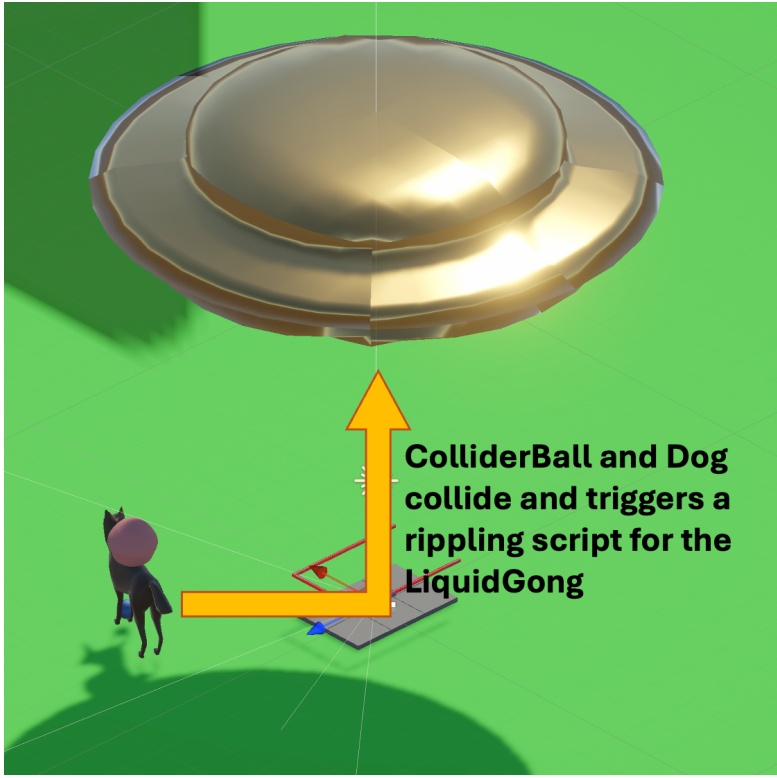
ReactRipple.cs:

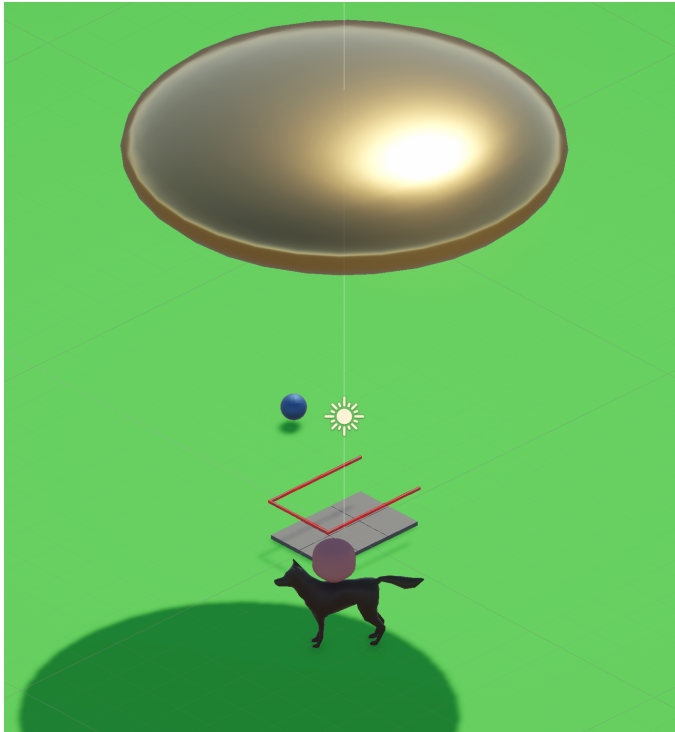
For the part of the script relevant to this project, I created a script that reacts to the collision of the Dog and Sphere objects and causes a ripple effect. Like the previous TimeRipple, this script uses a localEpicenter input to determine the distance of sphere vertices from the epicenter and changes the position of the vertices based on the sine-modified wave float. However, there are a few additions to this script. First, since it is no longer continuous, there is a isRippling bool added to create a reaction to the CollisionDetection script created by Gemini. This means that the ripple only occurs when the Dog and Sphere objects collide. Additionally, there are rippleTimer and decay floats which affect the amplitude of the waves as it approaches the input duration of the rippling effect. These all come together to create a conditional, vertex-level mesh deformation building on the TimeRipple.cs script to deform the LiquidGong object.

3. Evidence of requirements

[Use this table to point to the exact evidence for each requirement. Include video timestamps and screenshot labels.]

Requirement	Where it is demonstrated	Screenshot
Continuous Deformation	 [Unity Simulation; Screenshot 3: DogBallSphere_FP_Labelled.png]	DogBallSphere_FP_Labelled.png

Continuous Deformation	 ColliderBall and Dog both increase and decreases in size with time [Unity Simulation; Screenshot 4: DogBall_Iso_Labelled.png]	Screenshot 4: DogBall_Iso_Labelled.png
Input-responsive Deformation	 ColliderBall and Dog collide and triggers a rippling script for the LiquidGong [Unity Simulation; Screenshot 1: LiquidGong_Ripple_Iso_Labelled.png]	LiquidGong_Ripple_Iso_Labelled.png

Spatial placement	 [Unity Simulation; Screenshot 1 (in Screenshots_Unedited folder): LargeBall_SmallDog.png]	LargeBall_SmallDog.png
-------------------	---	------------------------

5. Assets, data, and citations

List every external asset, dataset, model, texture, sound, library, or reference used. State whether it was created by you, provided by the course, or obtained elsewhere. Provide a source link or formal citation where appropriate.

Item	Source / citation	How it was used
Roost Rails	Course-provided Asset	Used as a Roost Rails object for the player to stand on

6. AI-use disclosure

If you used generative AI during project development, disclose it below. If you did not use AI, write: No generative AI was used in project development.

Tool	Used for	What was incorporated, changed, or rejected
Gemini	Creation of CollisionDetection script	Incorporated wholly as a script to learn how collision detection works in Unity.

Gemini	Helped me create the Collision detection logic used in ReactRipple.cs	Incorporated learned understanding of collision detection logic in Unity.
Gemini	C# learning in general	Incorporated as I did not know how to use C# prior to doing this assignment.

7. Reflection

For the parts of the project that we were being graded on, it was fun and it helped me gain a greater understanding of mesh deformations and script-making. I really enjoyed implementing a way to implement an epicenter for my sine wave ripples in both TimeRipple.cs and ReactRipple.cs. It was very rewarding to see the models move in the way I intended after a while of working.

However, what made the project take a while was not so rewarding. What was very frustrating was the lack of compatibility with the XR Rig. The final iteration of my project was not my original idea: I was going to create a scene where one could inscribe into a plane so that someone could sculpt their own models in VR. But, since this script utilized interactions between the XR Rig's rays and the object's XR Simple Interactor, I couldn't implement it when I finally got to go to the Reality Roost. Since I couldn't find any documentation online or on the class Canvas page on how to do VR interactions without the XR rig, this left me scrambling to create a new scene that took more time and ultimately concluded in my late submission and less-interesting-than-anticipated scene.

This is not to say that the fault is the lack of XR Rig compatibility; I should have come in earlier and checked if my project worked in VR before coming in to record my project (and I'm sure its listed somewhere in the documentation that the rig and not just the camera are not compatible with the Reality Roost). So, next assignment I will work to improve my foresight and the thoroughness through which I read the documentation. Additionally, as I had seen some student succeed in getting their VR interactions to work with their scenes without the XR Rig (shoutout to Luke Ritter who stuck with me until 2:00 am on Friday and has easily one of the coolest looking and implemented first assignments I've seen), I would like to learn how to implement VR controller interactions without use of XR's given interactors that are primarily interact using the rig. Finally, I would like to be able to create a more whole scene with more time and love given to it. Currently, my first assignment scene only has 4 objects outside of the Reality Roost prefab and for my next assignment I'd like to make my scene more fun to be in and interact with.